Technical project report

MVP E-commerce API

A Node.js REST API for Product and Inventory Analytics

Arya Rad ajbisheh

Last update: October 23, 2026

The project Overview : 3

System Architecture and Design : 4

Data model and Database : 5

API Design and Query Processing : 8

Inventory Analysis and Business Logic : 11

Deployment : 14

How You Can Test the Project : 16

Contact and Feedback : 17

**Click a section number to jump to that section.*

The project Overview :

This project is a REST API for managing products and analyzing inventory in a grocery store. It was developed using Node.js, Express.js, MongoDB, and Mongoose. The **main features** include creating, updating, retrieving, and deleting product records, as well as filtering, sorting, and pagination. The API also provides inventory analysis features, such as calculating stock value by category or brand, comparing brands, classifying products based on ratings, and finding expired or soon-to-expire products.

The application follows the **MVC architecture**, with separate routes, controllers, and models. This structure helps keep request handling, application logic, and data operations organized and separate.

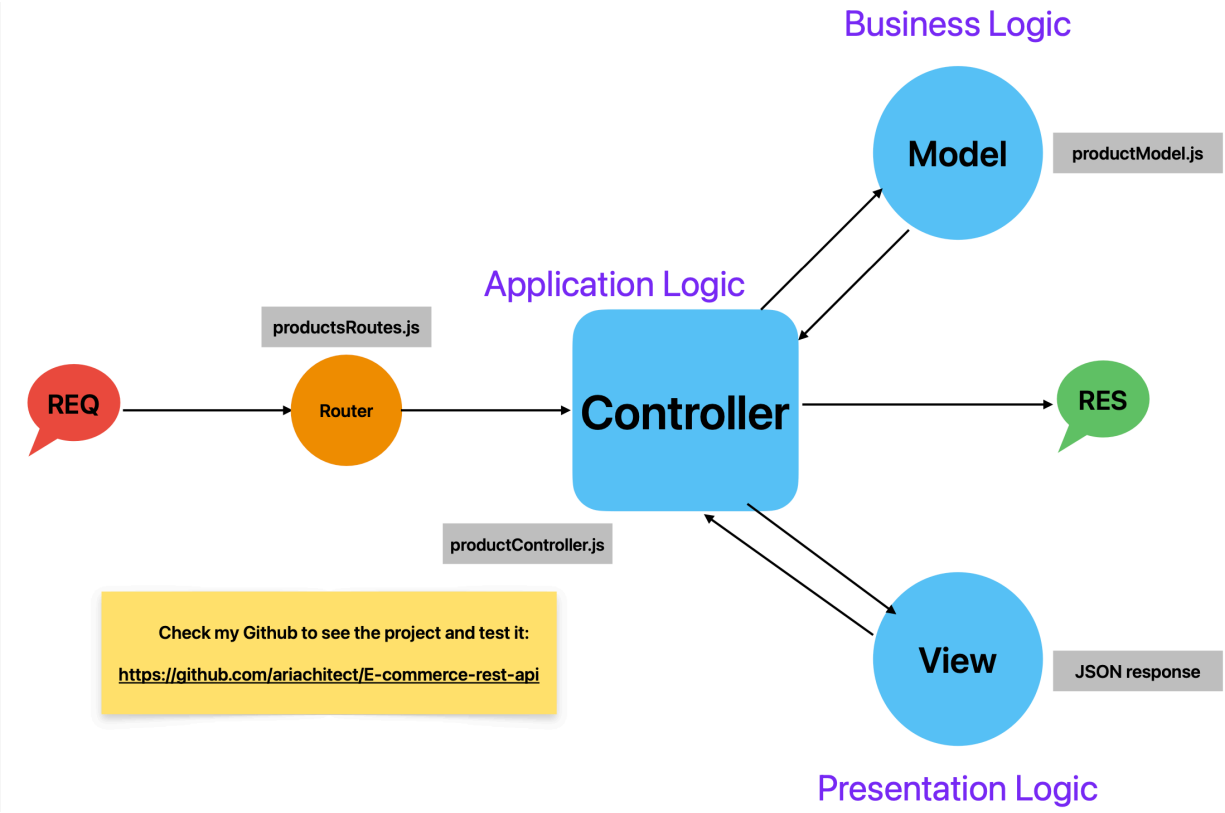
The project is **deployed** on Render, its source code is available on GitHub, and its API endpoints are **documented** in Postman, making it possible to explore and test its features.

My main goal with this project was to apply my academic knowledge and improve my software development skills after graduation.

System Architecture and Design :

I used the MVC (Model, View, Controller) architecture to organize the application and **separate** its different responsibilities. The routes define the available API endpoints, the controllers handle incoming requests and prepare responses, and the models manage data operations with MongoDB.

This structure also helps keep **business logic** separate from **application logic**. Instead of putting all the code in one place, I organized the code into different parts, each with its own responsibility. This makes the application easier to understand, maintain, and extend.



When a request (REQ) arrives, the router in `productsRoutes.js` directs it to the appropriate controller function in `productController.js`. The controller handles the request and, when needed, communicates with the model in `productModel.js` to retrieve or modify data in MongoDB and apply the required business logic. The model returns the result to the controller, which prepares the response.

Finally, the controller sends the response back to the client in JSON format. In this project, the View layer is represented by the JSON response rather than a separate user interface.

Data model and Database :

Database Technology:

MongoDB, a NoSQL document-oriented database, was selected for this project because its flexible document structure is suitable for storing product information with multiple attributes.

Object Data Modeling with Mongoose:

Mongoose was used as an Object Data Modeling (ODM) library to provide an abstraction layer between the application and MongoDB. It allows the application to define data schemas and interact with database documents through JavaScript objects and models, supporting a more structured and object-oriented approach to data management.

Product Schema:

The product data model was defined using a Mongoose schema to specify the **structure**, **data types**, **default values**, and **validation rules** of product documents.

Schema specification:

Field	Data type	Purpose
name	String	Product name
category	String	Product category
numInStock	Number	Available stock quantity
productPrice	Number	Product price
brand	String	Brand information
avgRating	Number	Average product rating
ratingQuantity	Number	Number of ratings
description	String	Product description
productionDate	Date	Production date
expirationDate	Date	Expiration date
images	Array of Strings	Image references
special	Boolean	Special product flag
priceDiscount	Number	Discount amount

Schema Validation and Constraints:

Mongoose allowed me to define validators in the product schema to check product data before saving it to the database.

I used built-in validators, such as `required`, `min`, `max`, `minlength`, `maxlength`, and `enum`, to check required fields, numeric ranges, string lengths, and valid product categories.

I also used a custom validator to ensure that the discount amount does not exceed 80% of the product price.

```
name: {
  type: String,
  required: [true, 'A product must have a name!'],
  minlength: [2, 'Product name is too short'],
  maxlength: [40, 'Product name is too long']
}
```

This example shows how Mongoose validators make the product name required and limit its length to between 2 and 40 characters.

```
priceDiscount: {
  type: Number,
  default: 0,
  validate: {
    validator: function(value) {
      return value <= this.productPrice * 0.8;
    },
    message:
      'The price discount should not exceed 80% of the product price'
  }
}
```

The custom validator function compares the discount value with 80% of the product price using this.productPrice and returns true if the condition is satisfied; otherwise, validation fails.

Additional Model Features:

The product model includes a **virtual property** to identify expired products and a custom instance method to detect products approaching expiration based on category-specific thresholds defined in a separate configuration file (/config/expirationPolicy.json).

category	Days Until Expiration
"dairy"	2
"bakery"	20
"beverages"	2
"snacks"	2
"fruits"	20
"vegetables"	2
"meat"	2
"seafood"	20

category	Days Until Expiration
"frozen_foods"	60
"personal_care"	60
"canned_goods"	60
"spices_and_condiments"	60
"grains_and_pasta"	60

```
// instance methode (documents will inherit this)
productSchema.methods.detectExpirSoon = function() {
  const today = new Date();
  const DaysTillExp = Math.ceil((this.expirationDate - today) / 86400000);
  return DaysTillExp >= 0 && DaysTillExp <= expPolicy[this.category];
};
```

The method calculates the remaining days by subtracting the current date from the product's expiration date and converting the result from milliseconds to days. It then compares the result with the category-specific threshold retrieved from `table above`.

API Design and Query Processing :

Following the MVC architecture, I implemented controllers to handle incoming requests **(req)** and return appropriate responses **(res)**. The API follows REST principles, providing a **standard** way for clients to access and interact with application resources.

REST allows me to hide internal implementation details and expose resources through endpoints. This makes the API easier to use because clients do not need to know how the system works internally. It also reduces the exposure of internal implementation details.

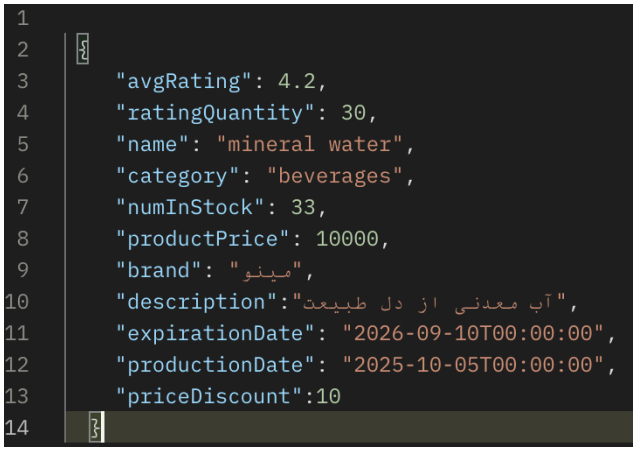
- The API uses the `/api/v1` prefix for versioning.
- Currently, product-related operations are available through `/api/v1/products`, while User and Order resources are planned for future development.

Routing:

I defined product routes in a separate router module and mounted it at `/api/v1/products`. Each route maps an HTTP method and path to its corresponding handler function in productHandler.

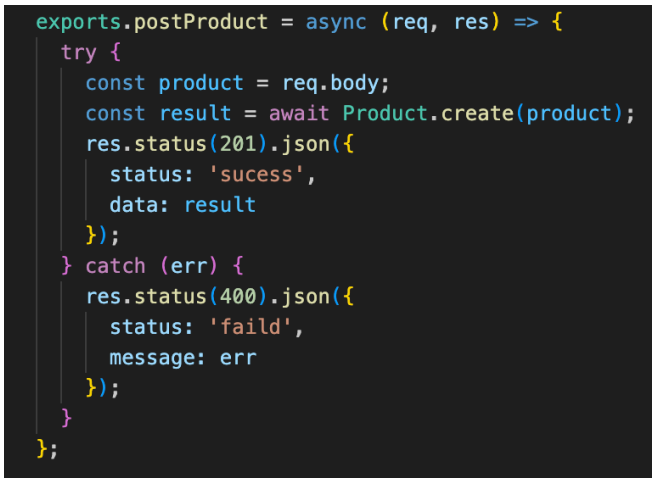
Request Handling Flow:

Clients can perform **CRUD** operations by sending standard HTTP requests to the corresponding API endpoints.



```
{
    "avgRating": 4.2,
    "ratingQuantity": 30,
    "name": "mineral water",
    "category": "beverages",
    "numInStock": 33,
    "productPrice": 10000,
    "brand": "مینو",
    "description":"آب معدنی از دل طبیعت",
    "expirationDate": "2026-09-10T00:00:00",
    "productionDate": "2025-10-05T00:00:00",
    "priceDiscount":10
}
```

For example, a client can send a `POST` request to `/api/v1/products` with a JSON body containing the product data.



```
exports.postProduct = async (req, res) => {
  try {
    const product = req.body;
    const result = await Product.create(product);
    res.status(201).json({
      status: 'sucess',
      data: result
    });
  } catch (err) {
    res.status(400).json({
      status: 'faild',
      message: err
    });
  }
};
```

When the request reaches the server, the router identifies the matching endpoint and forwards the request to the corresponding controller handler, which processes the request and returns an appropriate HTTP response to the client.

Advanced Query Features:

I used URL query **parameters** and **Mongoose** features to implement advanced query capabilities, allowing clients to filter, sort, select, and paginate results, as well as limit the number of returned documents.

- I defined these features in a separate `APIFeatures` module inside the `utils` directory and imported it into the controllers.
- This separation keeps the controllers cleaner, avoids repeating query logic, and makes the code easier to maintain and reuse.

Real-World Example:

Consider a store manager who wants to find dairy products priced below 30,000, with at least 3 units in stock and an average rating of 3.5 or higher. The manager wants to see products with earlier expiration dates first, followed by their prices, and only needs a few specific fields for comparison.(A complicated question !)

By setting the appropriate URL query parameters, the manager can easily retrieve the desired results on the first page, limited to 10 products, without needing to modify the application or write a separate query.

```
https://e-commerce-rest-api-508x.onrender.com/api/v1/products?category=dairy&produ
ctPrice[lt]=30000&numInStock[gte]=3&avgRating[gte]=3.5&sort=expirationDate,product
Price&fields=name,brand,productPrice,numInStock,avgRating,expirationDate&page=1&li
mit=10
```

```
{
  "status": "success",
  "result": 2,
  "data": [
    {
      "_id": "6ad9efc462c2e06d52fe8976",
      "name": "Strained Yogurt",
      "numInStock": 70,
      "avgRating": 3.6,
      "productPrice": 22636,
      "brand": "زرماکارون",
      "expirationDate": "2026-10-25T20:30:00.000Z",
      "expierd": false,
      "id": "6ad9efc462c2e06d52fe8976"
    },
    {
      "_id": "6ad9efc462c2e06d52fe8969",
      "name": "Whole Milk, 1 Liter",
      "numInStock": 4,
      "avgRating": 3.5,
      "productPrice": 20167,
      "brand": "کالـه",
      "expirationDate": "2026-10-26T20:30:00.000Z",
      "expierd": false,
      "id": "6ad9efc462c2e06d52fe8969"
    }
  ]
}
```


Note: You can find more details and examples of this query and other API endpoints in my API documentation. You can also test the API yourself using my Postman collection or clone my GitHub repository and run the project locally by following the instructions in the README.

Inventory Analysis and Business Logic :

I used Mongoose **Aggregation Pipelines** to process and analyze product data and develop features that provide useful information about inventory and product performance.

An aggregation pipeline processes documents through a series of stages, where each stage performs a specific operation, such as filtering, grouping, sorting, or calculating values. By combining these stages, I implemented features that go **beyond basic data retrieval** and provide **more useful insights** into the stored data.

```
exports.WarehouseValue = async (req, res) => {
  try {
    const { groupBy = 'category' } = req.query;
    //validator
    if (!['category', 'brand'].includes(groupBy.toLowerCase())) {
      throw new Error('Group id should be brand or category !');
    }
    const data = await Product.aggregate([
      {
        $group: {
          _id: `${groupBy}`,
          totalNumInStock: { $sum: '$numInStock' },
          totalValue: {
            $sum: { $multiply: ['$numInStock', '$productPrice'] }
          }
        }
      }
    ]);
    let totalWarehouseValue = 0;
    data.forEach(el => {
      totalWarehouseValue += el.totalValue;
    });
    res
      .status(200)
      .json({ status: 'success', TotalValue: totalWarehouseValue, data });
  } catch (err) {
    res.status(400).json({ status: 'failed', message: err.message });
  }
};
```

Aggregation-Based Features:

1. ***Top Five Products*** : Retrieves the top five products based on `avgRating` and `ratingQuantity`.
2. ***Warehouse Inventory*** : Value Calculates the total value of the warehouse inventory and provides inventory value totals for each product category or brand.
3. ***Product Classification (`ProdClass`)*** :provides inventory analytics by grouping products into classes such as A, B, and C.
4. ***Top Brands by Category (`bestBrands`)*** : Retrieves the top brands within a specified product category. The category name is passed as a URL parameter.

- Route Aliasing: I used route aliasing to provide shorter and more convenient endpoints for selected features. See my API documentation for details.

Implementation Example:

I chose the Warehouse Inventory Value feature because calculating the total value of stored products is a common task in real-world inventory management, and it requires more than basic query operations.

- Features like **sort** and **limit** can organize or reduce the results, but they cannot calculate the total warehouse value or the value of each product category on their own.

To implement this feature, I needed to group products by category (or brand) and perform calculations on the grouped data. This makes it a good example of how aggregation pipelines can be used to perform more complex data analysis.

Let's take a closer look at the code, how the function works, and what the output looks like:
The function allows the user to group products by either category or brand using the `groupBy` URL parameter.

- If no parameter is provided, products are grouped by category by default.

The aggregation pipeline uses `$group` to calculate the total stock quantity and inventory value for each group. The inventory value is calculated by multiplying each product's `numInStock` by its `productPrice` and summing these values. After the aggregation, the function adds up the inventory values of all groups to calculate the total warehouse value and returns it along with the grouped results in a JSON response (next page).

```
{
  "status": "success",
  "TotalValue": 137339563,
  "data": [
    {
      "_id": "canned_goods",
      "totalNumInStock": 192,
      "totalValue": 7205642
    },
    {
      "_id": "dairy",
      "totalNumInStock": 176,
      "totalValue": 9213968
    },
    {
      "_id": "grains_and_pasta",
      "totalNumInStock": 145,
      "totalValue": 9986604
    },
    {
      "_id": "bakery",
      "totalNumInStock": 210,
      "totalValue": 6073992
    },
    {
      "_id": "beverages",
      "totalNumInStock": 211,
      "totalValue": 12466510
    },
```


Note: For the full results and more information about this feature and the others, see my API documentation.

Deployment :

I used **Git** for version control to track changes during development and make it easier to maintain and extend the project in the future. I also published the code in a **GitHub** repository to make the project accessible and presentable.

After implementing the main features and reaching a suitable stage of development, I **deployed** the application on Render.

- This allowed me to gain practical experience with deploying a web application and provide an online demo of the API.

I configured Render to automatically redeploy the application whenever changes are pushed to the GitHub repository. This keeps the deployed version up to date with the latest code.

Testing and API Documentation :

I used **Postman** to **test** and **document** my API in an integrated environment.
It allowed me to :

- send HTTP requests
- inspect responses
- verify the behavior of different endpoints during development

After deploying the project on Render, I published my Postman collection online. This allows anyone with the collection link to access the available API requests and test the deployed endpoints without having to set up or run the project locally.

I also published complete API documentation covering all available endpoints, including example requests and responses, possible query parameters, and the necessary explanations. This makes the API easier to understand, test, and use.

What I Learned:

Working with **Git** and **GitHub** helped me :

- understand the importance of version control
- tracking changes
- maintaining a project throughout its development

Deploying the application on **Render** :

- gave me practical experience in making a web application publicly accessible.
- keeping the deployed version up to date through automatic redeployment.

Using **Postman** helped me:

- improve my API testing
- improve my documentation skills

publishing the collection and documentation taught me:

- the importance of making software accessible and understandable to other developers.

How You Can Test the Project :

You can test the project in three ways. The easiest option is to open my public Postman collection, where you can find the API endpoints, their descriptions, and ready-to-use requests to test the API.

- **Live API:** server_URL
- **Postman Collection:** Test the API using Postman
- **API Documentation:** View the API documentation
- **Github repository**: See the code

Alternatively, you can use any API testing tool you prefer with the live API URL and the API documentation above.

- Note: Due to Render's free-tier policies, the server may go to sleep after a period of inactivity. As a result, the first request may take up to 50 seconds to receive a response. Subsequent requests should be processed much faster.

If you prefer to run the project **locally**, you can access the source code through my GitHub repository. The README file and project description provide detailed, step-by-step instructions for setting up and running the application on your own system.

Contact and Feedback :

I would be grateful for any questions, feedback, or suggestions you may have regarding the project.
If you would like any further clarification or notice any issues, please feel free to contact me. I would sincerely appreciate the opportunity to learn from your feedback.

Please feel free to contact me through any of the following channels:

- Email: ariarad.ce@gmail.com
- GitHub: https://github.com/ariachitect